

FAST School of Computing
National University of Computer and Emerging Sciences
Islamabad Campus

EE-2003: Computer Organization and Assembly Language

PROJECT STATEMENT

Brick Breaker Game in 8086 Assembly Language

Graphics-Mode Interactive Game | INT 10h | x86 MASM

Course Code	EE-2003
Degree Program	BS Software Engineering
Semester	4th Semester, Spring 2026
Credit Hours	3 + 1 (Theory + Lab)
Project Weight	10% of Total Grade
Maximum Group Size	4 Students
Total Project Marks	100 Marks (across 3 iterations)

Spring 2026

1. Project Overview

This project requires you to design and implement a fully functional Brick Breaker game using 8086 Assembly Language in VGA graphics mode. The game must run on the MASM assembler targeting real-mode x86 architecture, utilizing BIOS interrupt services (primarily INT 10h) for all graphical rendering. No high-level language or external game library may be used.

The Brick Breaker game is a classic arcade genre where the player controls a horizontal paddle at the bottom of the screen. A ball bounces around the play area, breaking bricks arranged in a grid near the top of the screen. The player must prevent the ball from falling below the paddle. The game ends when all bricks are cleared (player wins) or when all lives are exhausted (game over).

This project directly assesses CLO-3 (Apply knowledge of Intel x86 architecture to develop moderately complex and well-modularized assembly programs) and CLO-4 (Analyze processor performance enhancement). Students must demonstrate mastery of procedures, the stack, memory addressing, graphics interrupts, real-time input handling, and program control flow—all covered in the course.

1.1 Why This Project?

A game is an ideal assembly project because it forces you to work within strict real-time constraints, manage memory directly, understand the coordinate system of VGA graphics mode, write modular and reusable code through procedures, and reason carefully about program state. These skills cannot be acquired by copying a solution—every detail must be understood to pass the viva.

1.2 Technology Stack

Assembler	MASM (Microsoft Macro Assembler) — version 6.x or compatible
Architecture	Intel 8086/x86 Real Mode
Graphics Mode	VGA Mode 13h (320x200, 256 colors) via INT 10h, AH=00h
Input Handling	BIOS Keyboard Interrupt (INT 16h) or direct port polling (60h)
Timing/Delay	Delay loops or BIOS timer interrupt (INT 15h / INT 1Ah)
Emulator / IDE	DOSBox + MASM, or emu8086 (students may choose)
Memory Model	SMALL model (.MODEL SMALL, .STACK, .DATA, .CODE)

1.3 VGA Mode 13h — Essential Background

Your entire game will run in VGA Mode 13h. Students must understand this mode before writing a single line of game code.

Mode 13h provides a 320×200 pixel resolution with 256 available colors. The video memory for this mode is mapped starting at segment address 0xA000 (i.e., far pointer A000:0000). Each pixel occupies exactly one byte at offset = (row $\times$ 320) + column. Writing a value to that byte address sets that pixel to the corresponding color index. Reading back from that address returns the current color at that pixel—which is how pixel-based collision detection works.

Do NOT use INT 10h, AH=0Ch (set pixel) in a fast game loop — it is too slow. Instead, write directly to video memory at segment A000h for performance. INT 10h, AH=0Ch is acceptable for initial testing but must be replaced in the final working game.

To enter Mode 13h:

```
MOV AX, 0013h    ; Set video mode to 13h (320x200, 256 colors)
INT 10h          ; BIOS video interrupt — activates graphics mode
```

To return to text mode (e.g., when exiting the game):

```
MOV AX, 0003h    ; Set video mode to 03h (80x25 text mode)
INT 10h
```

2. Group Policy and Academic Integrity

2.1 Group Formation

- Each group must consist of a maximum of four (4) students. Groups exceeding this limit will not be evaluated.
- Group formation is the responsibility of the students. Once a group is finalized and submitted, no changes will be permitted.
- A student not registered in any group by the deadline will receive zero marks for the project.

2.2 Equal Participation

This is a collaborative project. All members are expected to contribute equally across all iterations. The following rules apply during evaluation:

- Any group member may be asked to explain any part of the code or design—regardless of who wrote it.
- Inability to explain your group's code is treated as non-participation and will result in zero marks for that individual's viva component.
- The instructor and teaching assistants reserve the right to interview any student at any time during or after submission.

2.3 Plagiarism Policy

Plagiarism in this project will result in an F grade in the entire course for all involved parties. This includes copying code from other groups, from online repositories, or from AI tools (ChatGPT, Claude, Gemini, GitHub Copilot, etc.). Your code must be written and understood by your own group.

2.4 Use of AI Tools

You are not permitted to use any AI code generation tool to write, complete, or significantly modify your assembly code. These tools do not understand your specific game state, memory layout, or the exact course constraints—and any code they generate will be immediately apparent during viva when you cannot explain design decisions.

You may use AI tools for conceptual understanding—for example, to understand how VGA memory addressing works, or what a specific BIOS interrupt does. You must still implement everything yourself.

2.5 Submission Deadlines

- Deadlines are strict. No extensions will be granted under any circumstances.
- Submissions received 1–2 days late will incur a mark deduction as specified in the course policy.

- Submissions more than 2 days late will receive zero marks.

3. Game Design Specification

This section defines what your game must be. Read it carefully before designing anything. Your implementation must conform to these specifications—deviations must be justified in your documentation.

3.1 Screen Dimensions and Coordinate System

Your game will run in VGA Mode 13h: 320 pixels wide × 200 pixels tall. The origin (0, 0) is the top-left corner. X increases to the right; Y increases downward. Pixel color values are standard VGA palette indices (0–255).

All game element positions must be stored as integers in the .DATA segment. The recommended layout is as follows—you must adapt this to your own design:

Element	Suggested Region	Notes
HUD Bar	Y: 0–14	Score, lives, level display area (15px tall)
Play Field	Y: 15–195	Ball, paddle, bricks operate here
Brick Area	Y: 20–80	Brick grid rows (top portion of play field)
Paddle	Y: 185–190	Near the bottom; 5px tall, ~50px wide
Ball	Anywhere in play field	Approx. 4×4 px solid square recommended

These are suggested coordinates, not mandatory. You have creative freedom in the layout. However, all elements must be clearly visible, non-overlapping, and properly separated. The HUD must never overlap the play field.

3.2 The Brick Grid

Bricks are rectangular blocks arranged in rows and columns. The following rules apply:

- Each brick occupies a fixed rectangular region of the screen (e.g., 18px wide, 8px tall, with a 2px gap between bricks).
- The brick layout is stored as a 2D array in memory. A value of 1 indicates the brick is alive; 0 indicates it has been broken. You must design this data structure yourself.
- When the ball hits a brick, that brick's memory entry must be set to 0, the brick must be erased from the screen (overwritten with background color), and the score must be updated.
- Different levels must have different brick layouts. You may vary: number of rows, colors of bricks, presence of 'strong' bricks (requiring multiple hits in Level 3), or arrangement patterns.
- Brick colors should vary by row or level to make the game visually informative.

3.3 Ball Mechanics

The ball is defined by its current X and Y position and its current direction vector (dX and dY). These four values must be stored as signed integers in memory.

The ball moves by adding dX to its X position and dY to its Y position each game loop iteration. Initially, both dX and dY should be non-zero (e.g., dX = +2, dY = -2). The following collision behaviors must be implemented:

- Left/Right wall collision: When the ball's X reaches 0 or 319 (adjusted for ball size), reverse the sign of dX.
- Top wall collision: When the ball's Y reaches 15 (top of play field), reverse the sign of dY.
- Paddle collision: When the ball overlaps the paddle region, reverse the sign of dY. You may optionally add angle variation based on where the ball hits the paddle—this is encouraged but not required.
- Brick collision: When the ball pixel overlaps a brick region that is marked as alive, destroy the brick and reverse dY (and potentially dX depending on collision side).
- Bottom wall miss: When the ball's Y exceeds 195 (below paddle), a life is lost. Reset the ball to the center and reduce lives by 1.

Collision Detection Hint: The simplest approach is pixel-color detection. After computing the ball's new position, read the pixel color at that position from video memory (segment A000h). If the color matches a brick color, a brick was hit. If it matches the paddle color, the paddle was hit. This avoids complex geometric intersection math.

3.4 Paddle Mechanics

The paddle is a horizontal rectangle near the bottom of the screen. Its position is defined by its left-edge X coordinate. The paddle width is fixed for a given level.

- The paddle moves left and right in response to keyboard input (Left/Right arrow keys or A/D keys).
- The paddle must not move outside the screen boundaries (X must remain between 0 and 320 minus paddle width).
- Each keypress moves the paddle by a fixed number of pixels (e.g., 4px per step). The step size must not be so large that the paddle 'jumps' visually.
- After moving, the paddle must be redrawn: erase the old position (paint it with background color) and draw the new position.

3.5 HUD (Heads-Up Display)

The HUD must be displayed at the top of the screen and must update in real-time every game loop iteration. Required HUD elements:

- Score — displayed as a numeric value, updated whenever a brick is broken.
- Lives — displayed as either a number or as graphical symbols (hearts, stars, etc.). Starts at 3.

- Level — displays the current level number.
- Player Name — the name entered at the name input screen, displayed as a string.

For rendering text in graphics mode, you must implement a character drawing routine manually (INT 10h, AH=09h writes to text mode—it cannot be used in Mode 13h). You must either use BIOS font data or define your own small bitmap font for numeric characters.

Technique for text in Mode 13h: BIOS stores an 8x8 font table starting at INT vector 43h (segment:offset stored at address 0000:010Ch). You can read this font table and use it to blit characters directly to video memory. This is covered indirectly in the graphics lab sessions.

4. Required Screens

Your game must implement all six screens listed below. Each screen must be a separate, clearly structured section of your code—ideally as a procedure or set of procedures.

4.1 Home Screen

The first screen shown when the program starts. It must display:

- The game title: BRICK BREAKER (or a custom title of your choice, rendered in large styled text)
- A background design using colors and patterns — not a plain black screen
- An instruction prompt at the bottom: Press any key to continue
- The program waits here until the user presses any key, then transitions to the Name Input Screen

Design Expectation: The Home Screen is the first impression of your game. Use color fills, border designs, and spacing deliberately. Examine classic arcade games for inspiration. Plain text on a black background will not score well on the design component.

4.2 Name Input Screen

This screen collects the player's name. Requirements:

- A visible input box drawn on screen (border + fill rectangle)
- A prompt label: Enter your name:
- Characters typed by the user appear inside the box in real-time
- Backspace key deletes the last entered character
- Enter key confirms the name and moves to the Main Menu
- Name must be stored in a memory buffer in the .DATA segment — it will be reused throughout the game in the HUD
- Input must be limited to a maximum of 12–15 characters to avoid overflow

You must implement this without using any DOS/BIOS string input interrupt. Read keyboard input one character at a time using INT 16h, AH=00h (blocking keyboard read) and manage the string buffer manually. This is an explicit requirement.

4.3 Main Menu Screen

The main navigation hub. Options to display:

- Start Game
- Instructions
- High Scores
- Exit

Navigation requirements:

- The currently selected option must be visually highlighted (e.g., different color, arrow indicator, or background highlight).
- Up/Down arrow keys move the selection cursor.
- Enter key activates the selected option.
- The menu must be reachable from Game Over and Win screens (return to menu option).

4.4 Instructions Screen

A static informational screen explaining:

- Controls: Left/Right arrow or A/D to move paddle
- Objective: Break all bricks to advance to the next level
- Lives: You start with 3 lives. Losing the ball costs one life.
- Bonuses: Brief explanation of the 3+ bonus types (added in Iteration 3)
- Press any key or Escape to return to Main Menu

4.5 High Scores Screen

Displays the top scores stored in the game. In Iteration 1, static/dummy data is acceptable. In Iteration 3, if file handling is implemented (bonus), actual saved scores should appear here.

- Format: Rank | Name | Score (three columns)
- Display at least 5 entries (can be hardcoded in Iteration 1)
- Visual formatting must be clear and aligned — columns must be visually distinct

4.6 Game Screen

The main gameplay screen. This contains the HUD area and the play field. In Iteration 1, only the static layout is required. In Iteration 2 and 3, it must be fully functional. All elements described in Section 3 (ball, paddle, bricks, HUD) must be present and correctly positioned.

5. Iteration 1: Graphics & UI Prototype [30 Marks]

Due: To be announced by course instructor

Iteration 1 focuses entirely on the visual foundation of the game. No game logic, ball movement, or collision detection is required at this stage. The goal is to build and connect all six screens with proper graphics and basic interactivity.

What counts as 'graphics' in this context: Using INT 10h or direct video memory writes to draw filled rectangles, borders, color fills, and text. A screen that only uses INT 10h text-mode output (AH=09h, 0Eh, etc.) without entering Mode 13h does NOT qualify as a graphics screen.

5.1 Marking Breakdown — Iteration 1

Home Screen (5 marks)

Component	Marks
Title rendered visibly and approximately centered	2
Background has design beyond plain black fill	1
Instruction text present (e.g., Press any key to continue)	1
Alignment and visual spacing — elements are not cramped	1

Name Input Screen (5 marks)

Component	Marks
Input box drawn with visible border and fill	1
Characters appear on screen as user types	1
Backspace deletes the last character correctly	1
Enter key confirmed — transitions to Main Menu	1
Name stored correctly in .DATA memory buffer	1

Main Menu Screen (8 marks)

Component	Marks
All four options displayed (Start, Instructions, High Scores, Exit)	2
Up/Down arrow navigation moves the selection	2
Selected option is visually highlighted or marked	2

Visual layout — aligned, clear, readable	2
--	---

Instructions Screen (4 marks)

Component	Marks
Controls explained (paddle movement keys)	1
Game rules described (objective, lives, ball)	2
Layout is readable — adequate spacing and contrast	1

High Scores Screen (3 marks)

Component	Marks
Score entries displayed (static data acceptable)	1
Format is tabular — rank, name, score columns	1
Visual structure — borders or separators present	1

Game Screen Layout (5 marks)

Component	Marks
Ball drawn as a filled shape in play field	1
Paddle drawn as a rectangle at bottom of play field	1
Brick grid drawn — rows and columns visible	1
HUD bar shows score, lives, name, and level placeholders	1
All elements are non-overlapping and correctly positioned	1

Code Quality and Documentation (5 marks)

Component	Marks
Code is divided into procedures (not monolithic)	2
Code is readable — consistent indentation, meaningful labels	1
Documentation explains each screen and the navigation flow	2

5.2 Iteration 1 Submission Requirements

1. Source code file (.asm) — complete, assembles without errors
2. Documentation (PDF or Word, minimum 5 pages) containing:
 - Description of each screen and its purpose
 - Diagram or description of screen navigation flow

- Explanation of how name input is implemented in memory
- 3. Video demonstration (2–5 minutes) showing:
 - All six screens rendered correctly
 - Navigation between screens working
 - Name input with backspace and enter working

5.3 What Is NOT Required in Iteration 1

DO NOT implement the following in Iteration 1: ball movement, collision detection, lives decrementation, score counting, bonus items, level transitions. Attempting these does not compensate for missing UI requirements. Screen design quality will be the sole basis of marks in this iteration.

6. Iteration 2: Functional Game — Level 1 [40 Marks]

Due: To be announced by course instructor

Iteration 2 converts your static prototype into a fully playable Level 1 game. All screens from Iteration 1 must remain functional. The game loop, collision system, lives system, and HUD updates must all be implemented and working.

Iteration 2 is the most technically demanding iteration. Ball physics, collision detection, and real-time rendering are non-trivial in assembly. Plan your time accordingly. Start with ball movement and wall collisions before tackling brick collisions.

6.1 The Game Loop — Mandatory Architecture

Your game must be structured around an explicit game loop. The loop must execute the following steps in this order on every frame:

Step 1	Read keyboard input (non-blocking: check if key is available before reading)
Step 2	Update paddle position based on input
Step 3	Erase the ball's current screen position (paint background color)
Step 4	Compute new ball position (add dX to X, add dY to Y)
Step 5	Perform collision checks: walls → paddle → bricks (in this order)
Step 6	Handle collision results: reverse direction, destroy brick, update score/lives
Step 7	Draw the ball at its new position
Step 8	Update HUD display (score, lives)
Step 9	Apply frame delay to control game speed
Step 10	Check for game-end conditions (all bricks gone OR lives = 0)
Step 11	Jump back to Step 1

Non-blocking keyboard input is critical. If you use INT 16h, AH=00h (blocking read) inside the game loop, the entire game will freeze waiting for a key. Use INT 16h, AH=01h (check buffer status) first, and only call AH=00h if a key is available.

6.2 Marking Breakdown — Iteration 2

Input Handling (5 marks)

Component	Marks
Paddle moves left/right in response to keyboard input	2
Menu navigation from Iteration 1 still works correctly	1
Name input from Iteration 1 still works correctly	1
No input lag, freezing, or skipped keypresses	1

Ball Mechanics (10 marks)

Component	Marks
Ball moves continuously without stopping or freezing	2
Ball reflects correctly off left and right walls	2
Ball reflects correctly off the top wall	1
Ball reflects correctly off the paddle surface	3
Ball detects brick collision and responds correctly	2

Paddle Mechanics (5 marks)

Component	Marks
Paddle movement is smooth and consistent	2
Paddle stops at left boundary ($x = 0$)	1
Paddle stops at right boundary ($x = 320 - \text{paddleWidth}$)	1
Paddle speed is reasonable — not too fast or too slow	1

Brick System (6 marks)

Component	Marks
Brick is erased from screen when hit by ball	2
Score increments by correct amount per brick broken	2
No false (ghost) hits — brick not already destroyed triggers no effect	2

HUD Display (4 marks)

Component	Marks
Score value updates in real-time as bricks are broken	1
Lives count decrements correctly when ball is missed	1
Player name (from name input) is displayed in HUD	1

Level indicator shows Level 1	1
-------------------------------	---

Lives System (5 marks)

Component	Marks
Game starts with exactly 3 lives	1
Life is decremented when ball passes below paddle	1
Ball resets to starting position and pauses briefly after life loss	1
Game Over state triggered when lives reach 0	2

Game Flow Integration (5 marks)

Component	Marks
Start Game from menu correctly launches Level 1 gameplay	2
Game Over screen is shown with final score when lives exhaust	1
Player can return to Main Menu from Game Over screen	1
Game does not crash, hang, or produce visual corruption	1

6.3 Iteration 2 Submission Requirements

4. Source code file (.asm) — fully assembles, Level 1 completely playable
5. Documentation (PDF or Word) adding to Iteration 1 report:
 - Game loop description — explain the order of operations
 - Collision detection explanation — how wall, paddle, brick hits are detected
 - Lives system explanation — how ball reset and life decrement work
6. Video demonstration (3–6 minutes) showing:
 - Menu navigation into Level 1
 - Paddle responding to keyboard
 - Ball bouncing off walls and paddle
 - Bricks being broken with score incrementing
 - Life lost when ball falls below paddle
 - Game Over screen appearing after 3 lives exhausted

6.4 What Is NOT Required in Iteration 2

The following features belong to Iteration 3. Do not attempt them in Iteration 2:

- Multiple levels (only Level 1 is required)
- Bonus / power-up system
- File handling (score saving/loading)
- Advanced visual effects

- Win screen (Level 1 cannot be 'won' in Iteration 2 — game ends at Game Over only)

7. Iteration 3: Full Game with Advanced Features [30 Marks]

Due: To be announced by course instructor

Iteration 3 extends the working Level 1 game into a complete, polished game experience. You will add two additional levels with progressive difficulty, a bonus/power-up system, a Win screen, and ensure overall game stability and quality. This is where your project becomes a complete game.

Iteration 3 builds directly on your Iteration 2 codebase. If your Level 1 game has bugs or crashes, they will cascade into Level 2 and Level 3. Fix all known Iteration 2 issues before adding new features.

7.1 Multi-Level System

You must implement three distinct levels. Level 1 is reused from Iteration 2. Level 2 and Level 3 must be new and must feel meaningfully different from Level 1. Differences can be achieved through:

- Increased ball speed (increase the magnitude of dX and/or dY)
- More rows or denser brick layouts
- Different brick color patterns
- Shorter paddle width
- Introduction of strong bricks (Level 3 only) that require 2–3 hits — track hit count per brick in the brick data array

Level progression rules:

- When all bricks in the current level are destroyed, display a brief Level Complete message, then load the next level's brick layout, reset ball position, and reset the timer.
- Player's accumulated score and remaining lives carry over between levels.
- After completing Level 3, the Win Screen is shown.

7.2 Bonus / Power-Up System

A minimum of three distinct bonus types must be implemented. Bonuses are spawned when a brick is destroyed and must fall downward toward the paddle.

General requirements:

- Bonus spawn is probabilistic — not every brick drop spawns a bonus (implement a simple random or counter-based trigger).
- Each bonus is represented as a small falling object on screen (e.g., a colored square with a letter identifier).
- If the bonus reaches the paddle, it is collected and its effect is applied.
- If the bonus falls below the paddle, it disappears with no effect.
- Only one bonus may be active on screen at a time in the minimum implementation.

Required minimum: implement any 3 of the following bonus types:

Bonus Type	Suggested Color	Effect Description
Slow Ball	Cyan (3)	Reduces ball speed (decrease dX/dY magnitude) for a limited time
Fast Ball	Red (4)	Increases ball speed — a risk/reward power-up
Extra Life	Green (2)	Adds +1 to the lives counter (max of 5 lives)
Wide Paddle	Yellow (14)	Increases paddle width by 50% for a limited time or for the rest of the level
Narrow Paddle	Magenta (5)	Decreases paddle width — a negative power-up (makes game harder)

A bonus that spawns but has no effect on gameplay will receive zero marks for that bonus type. The effect must be observable and correctly applied. Static bonuses or visual-only bonuses are penalized.

7.3 Marking Breakdown — Iteration 3

Multi-Level System (10 marks)

Component	Marks
Level 2 is implemented and playable	2
Level 3 is implemented and playable	2
Ball speed increases in each subsequent level	2
Brick layouts are visually different across levels	2
Level transition works smoothly — no crashes or data corruption	2

Bonus / Power-Up System (10 marks)

Component	Marks
Bonus spawn logic is triggered (random or counter-based)	2
Bonus falls downward with visible animation each frame	2
Paddle collision with falling bonus is correctly detected	2
At least 3 bonus types implemented with observable effects	4

Complete Game Flow (5 marks)

Component	Marks
Level 1 → Level 2 → Level 3 progression works correctly	2
Game Over screen appears with final score when lives exhaust	1
Win/Completion screen appears after clearing Level 3	1
Player can return to Main Menu or replay from end screens	1

Game Polish & Stability (5 marks)

Component	Marks
Game does not crash or freeze during normal play	2
Screen rendering is smooth — minimal flickering	1
Difficulty is balanced — game is challenging but playable	1
Screen transitions are clean and visually consistent	1

7.4 Iteration 3 Submission Requirements

7. Source code file (.asm) — 3 levels, bonus system, full flow working
8. Documentation (PDF or Word — final version, updated from previous iterations):
 - Level design: describe the differences between each level
 - Bonus system: explain how random spawn, falling, and collision are implemented
 - Complete game flow diagram
9. Video demonstration (5–8 minutes) showing:
 - Level 1 → Level 2 → Level 3 progression in one play session
 - Each bonus type being collected and its effect observed
 - Game Over scenario (3 lives lost)
 - Win scenario (all 3 levels cleared)

8. Bonus Features [+10 Marks Maximum]

Students who complete all three iterations successfully may earn additional marks by implementing the following advanced features. These are challenging and time-consuming — only attempt them after the mandatory features are fully working and stable.

Feature	Marks	Requirement
File Handling — Save & Load High Scores	+3	On Game Over / Win, save player name and score to a file. Load and display on High Scores screen. Must use DOS INT 21h file services.
Pause System	+2	A designated key (e.g., P) pauses the game — ball and bonuses freeze. Pressing the same key resumes. A PAUSED indicator must appear on screen.
PC Speaker Sound Effects	+2	Use port 61h and timer (8253/8254) programming to produce beep sounds on brick hit, life loss, level complete events. Must be distinctly different pitches.
Mouse Support	+2	Use INT 33h to read mouse X position and use it to control paddle position as an alternative to keyboard input.
Visual Effects	+1	Ball leaves a short trail (last 3–5 positions drawn in fading color). And/or a brief flash effect when a brick is destroyed.

8.1 Out-of-the-Box Enhancements (High Distinction)

Students who implement the following may be considered for exceptional recognition during project viva, at the discretion of the instructor. These are research-level features for 8086 assembly:

- **Multi-Ball Mode:** One specific bonus splits the ball into two simultaneous balls. Both must interact correctly with all game elements.
- **Moving Bricks:** In Level 3, certain bricks shift horizontally back and forth each few frames.
- **Boss Level (Level 3):** A single large brick requiring 5+ hits. Displays remaining hit count visually.
- **Combo System:** Consecutive brick hits within a short time increment a score multiplier (x1 → x2 → x3). Multiplier resets if the ball hits the paddle or a wall without breaking a brick in between.
- **Adaptive Difficulty:** Track average time between missed balls. If the player is performing well, gradually increase ball speed mid-level.

These features cannot be implemented without a deep and working understanding of your entire codebase. They are genuinely difficult. Attempting them prematurely risks breaking your working game. Only pursue these after fully completing all three iterations.

9. Marks Summary

Component	Marks	% of Project
Iteration 1: Graphics & UI Prototype	30	30%
Iteration 2: Functional Game — Level 1	40	40%
Iteration 3: Full Game + Advanced Features	30	30%
Bonus Features (optional)	+10	Up to +10%
TOTAL (mandatory)	100	100%

The 10-mark project component in the overall course grade is derived from this 100-mark project, scaled proportionally. For example, a score of 80/100 on this project contributes 8/10 marks toward your final course grade.

10. CLO and PLO Alignment

This project is aligned with the following course and program learning outcomes:

CLO	Project Coverage	How This Project Assesses It
CLO-1	All Iterations (screen setup, mode switching)	Students must correctly set up VGA graphics mode, understand BIOS interrupts, and demonstrate knowledge of how the CPU interacts with display hardware.
CLO-2	Iteration 1 (HUD, text output, I/O)	INT 10h, INT 16h, and direct port use are covered. Students must use assembly-language I/O primitives without high-level abstraction.
CLO-3	Iterations 2 & 3 (game loop, collision, bonuses)	Primary CLO. The game requires moderately complex, well-modularized assembly programs using procedures, the stack, memory addressing, and control flow.
CLO-4	Iteration 3 (multi-level speed, timing)	Students must reason about how timing and loop speed affect perceived performance, connecting to cache and pipeline concepts from theory.

11. Technical Guidance and Common Pitfalls

11.1 Recommended Code Structure

Your code must not be a single large monolithic procedure. The following organization is strongly recommended. You are free to name procedures differently.

Procedure / Label	Responsibility
INIT_GRAPHICS	Set video mode to 13h; initialize segment registers; clear screen
SHOW_HOME_SCREEN	Draw all elements of the home screen; wait for keypress
GET_PLAYER_NAME	Manage name input buffer; handle keyboard character loop
DRAW_MAIN_MENU	Draw menu options; highlight selected; handle navigation loop
LOAD_LEVEL	Initialize ball position/direction, brick array, paddle position for a given level number
GAME_LOOP	Main gameplay loop: calls input, update, draw, check-end procedures per frame
READ_INPUT	Non-blocking keyboard check; update paddle direction variable
MOVE_PADDLE	Apply direction to paddle X; clamp to boundaries; erase/redraw paddle
MOVE_BALL	Erase ball; add dX/dY to position; store new position
CHECK_COLLISIONS	Call wall, paddle, and brick collision sub-procedures
CHECK_WALL_COLLISION	Check X/Y against screen boundaries; reverse dX or dY as needed
CHECK_PADDLE_COLLISION	Read pixel at new ball position; if paddle color, reverse dY
CHECK_BRICK_COLLISION	Read pixel at new ball position; if brick color, destroy brick, update score
DESTROY_BRICK	Erase brick from screen; set brick array entry to 0; call UPDATE_SCORE
UPDATE_HUD	Redraw score, lives, level, and player name in HUD region
DRAW_BONUS	Handle active bonus falling; check paddle collision; apply effect
SHOW_GAME_OVER	Display Game Over screen with final score; wait for input
SHOW_WIN_SCREEN	Display Win/Congratulations screen; wait for input
EXIT_GAME	Restore text mode (Mode 03h); terminate program via INT 21h AH=4Ch

11.2 Common Mistakes and Penalties

Mistake	Consequence
Plain text-mode screens (no Mode 13h graphics)	Graphics marks cut entirely for that screen

No visual alignment — elements randomly placed	Layout/design sub-marks cut
Hardcoded static menu with no navigation	Navigation marks cut
Ball passes through bricks or paddle (no collision)	Collision marks cut directly
Paddle crosses screen boundary	Paddle boundary marks cut
Bricks not erased from screen when hit	Brick system marks cut
Score does not update in real-time	HUD marks cut
Game crashes or hangs during play	Flow/stability marks cut significantly
Same brick layout in all three levels	Level system marks cut
Bonus appears visually but applies no effect	Bonus marks cut to zero for that type
No win screen after clearing Level 3	Game flow marks cut
Monolithic code — no procedures	Code quality marks cut
Insufficient documentation	Documentation marks cut

11.3 Suggested Development Sequence

Students often get stuck because they try to build everything at once. The following sequence is recommended:

10. Learn Mode 13h: Write a test program that enters Mode 13h, draws colored rectangles, and exits.
11. Master direct video memory writes: Set a pixel at (100, 100) without using INT 10h, AH=0Ch.
12. Implement the game screen layout (Iteration 1 final deliverable).
13. Implement non-blocking keyboard input and paddle movement.
14. Implement ball movement with wall reflection (no brick collision yet).
15. Add paddle collision.
16. Add brick array data structure and brick collision.
17. Integrate score and lives.
18. Add Game Over flow.
19. Test all above thoroughly before adding Level 2.
20. Add Level 2 and Level 3 with varied layouts and ball speed.
21. Add bonus system after all levels are stable.
22. Polish, fix bugs, record video.

11.4 INT Reference Quick Sheet

Interrupt	Function	Key Registers
-----------	----------	---------------

INT 10h, AH=00h	Set video mode	AL=13h (Mode 13h) or AL=03h (text)
INT 16h, AH=00h	Read key (blocking)	Returns: AH=scan code, AL=ASCII
INT 16h, AH=01h	Check key buffer (non-blocking)	ZF=1 if no key; ZF=0 if key ready (do NOT consume it – call AH=00h after)
INT 21h, AH=4Ch	Terminate program	AL=return code (0 for clean exit)
INT 21h, AH=3Ch	Create file	CX=attribute, DS:DX=filename
INT 21h, AH=3Dh	Open file	AL=access mode, DS:DX=filename
INT 21h, AH=40h	Write to file	BX=handle, CX=bytes, DS:DX=buffer
INT 21h, AH=3Fh	Read from file	BX=handle, CX=bytes, DS:DX=buffer
INT 33h, AX=0000h	Reset mouse driver	Returns AX=FFFFh if mouse present
INT 33h, AX=0003h	Get mouse position and buttons	Returns CX=X, DX=Y, BX=button state

12. Documentation Requirements

Each submission must be accompanied by a documentation report. The same document is updated and expanded across all three iterations. The final submission (Iteration 3) must represent the complete, polished report.

12.1 Required Sections

- 23. Cover Page: Group name, member names and IDs, course code, university, semester, iteration number
- 24. Game Description: Brief overview of the game and its features
- 25. Screen Flow Diagram: A visual diagram showing how screens connect to each other
- 26. Technical Design (one section per iteration):
 - Iteration 1: Screen descriptions, color choices, navigation logic
 - Iteration 2: Game loop architecture, collision detection approach, lives system
 - Iteration 3: Level design rationale, bonus system logic, game flow
- 27. Data Structures: Describe all major .DATA variables — their type, purpose, and size
- 28. Procedure Descriptions: List all procedures with a 2–3 sentence description of each
- 29. Known Issues: Honestly list any limitations or bugs in your submission
- 30. Task Division: Table showing which group member was primarily responsible for which component

12.2 Documentation Quality

The documentation will be evaluated on clarity, completeness, and accuracy. It must accurately describe what you submitted — not what you intended to submit. Describing features you did not implement will be penalized.

Your documentation is also a demonstration of your understanding. If you can explain your collision detection approach clearly in writing, you will be able to answer viva questions about it confidently.

13. Final Game Flow Architecture

By the end of Iteration 3, your complete game must implement the following screen flow. This diagram must be reproducible from your code and documentation.

```

Program Start
|
v
[ Home Screen ] --> any key
|
v
[ Name Input Screen ] --> Enter key
|
v
[ Main Menu ]
|-- Start Game --> [ Level 1 ] --> [ Level 2 ] --> [ Level 3 ]
|                                     |                                     |
|                                     v (lives = 0)                         v (all
bricks cleared)
|                                     [ Game Over ] <--- [ Win Screen ]
|                                     |
|                                     +-----> return to [ Main Menu ] <-----+
|-- Instructions --> [ Instructions Screen ] --> [ Main Menu ]
|-- High Scores  --> [ High Scores Screen ] --> [ Main Menu ]
|-- Exit         --> [ Restore text mode ] --> Program End

```

14. A Final Note to Students

This is one of the most technically demanding projects you will encounter in your undergraduate curriculum. It requires everything you have learned in this course — and it will require you to learn additional things independently.

Assembly language programming at this scale is difficult. You will hit walls. The ball will not move. The screen will flicker. The game will crash. The key is to debug systematically: isolate one component at a time, test it in a minimal program, then integrate it. Do not write 300 lines of code and then try to find a bug.

Groups that start early, divide work intelligently, meet regularly, and hold each member accountable will produce good games. Groups that leave it to the last week will not.

The difference between a group that builds a working game and one that submits a static screen is almost never intelligence — it is almost always discipline and early start.

Good luck. Build something you are proud of.